SF

PROJECT PRESENTATION · 2026

Multi-tenant scraping & browser automation platform

ScrapeFlow

A complete, production-shaped web scraping API and dashboard — one durable job queue, a real headless browser path, SSRF-safe egress, tenant isolation, and artifact storage.

Python 3.12 | FastAPI | Celery + Redis | PostgreSQL 16 | Selenium Grid | S3 artifacts | Next.js 16 | React 19 | TypeScript

11	29	322	15
SERVICES IN DEV	API ROUTES	TESTS PASSING	DB MODELS

REPOSITORY
web-scrapping-tool / scrapeflow

BASE COMMIT
`0c6d96c`

EVIDENCE
Live stack · real scrape · 322 tests

DATE
10 November 2026

LIVE DEMO RECORDED ON
Docker Compose · 11 services · healthy

AGENDA

What this deck covers

Product → architecture → every technology choice, point by point → live artifacts → quality and honest limits.

01

Problem & solution

Why scraping at scale is hard, and the product that answers it.

02

Architecture

Control plane, workers, data stores and the request lifecycle.

03

Languages & package managers

Python 3.12 + `uv`; TypeScript + `npm`. Why each was chosen.

04

Backend libraries

FastAPI, SQLAlchemy, Celery, Selenium, boto3, security and ops.

05

Frontend libraries

Next.js 16, React 19, Tailwind 4, the BFF pattern and tooling.

06

Live artifacts

Dashboard screenshots, a real job, screenshot, HTML and JSON.

07

Infrastructure & quality

Docker topology, cross-platform runners, test and lint gates.

08

Verified vs unverified

Exactly what was proven locally, and what needs external services.

IN SHORT The deck follows the order of a code review: what it does, how it is wired, which library does what, then the evidence.

THE PROBLEM

Scraping at scale is not a `requests.get()`

A single script works once. A platform has to survive concurrency, hostile pages, tenants, and audits.

REALITY 01

Modern pages need a real browser

Client-side rendering, lazy images, cookie walls and bot checks mean raw HTML is often empty. Chrome must be pooled, timeboxed and reaped.

REALITY 02

Long jobs cannot live in a request

A 60-second browser run would block an HTTP worker. Work must be queued durably, retried, cancellable, and observable from outside the request.

REALITY 03

User-supplied URLs are an attack surface

Without validation, `http://169.254.169.254` or `localhost` turns a scraper into an internal network scanner. DNS rebinding defeats naive checks.

REALITY 04

Multi-tenancy is not optional

Every job, key, artifact, schedule and webhook must be scoped to an organisation, with roles, quotas, rate limits and an audit trail.

Implicit requirements: artifact storage with hashes and expiry, idempotent submissions, structured logs with correlation IDs, metrics an operator can alert on, and a UI that does not leak tokens.

Implicit constraints: the API process must never launch Chrome; workers may crash mid-job; storage may be MinIO, S3 or a mock; the same compose file should run on Linux, macOS and Windows.

IN SHORT The hard part is never the HTTP fetch — it is isolation, durability, safety and accounting around it.

THE SOLUTION

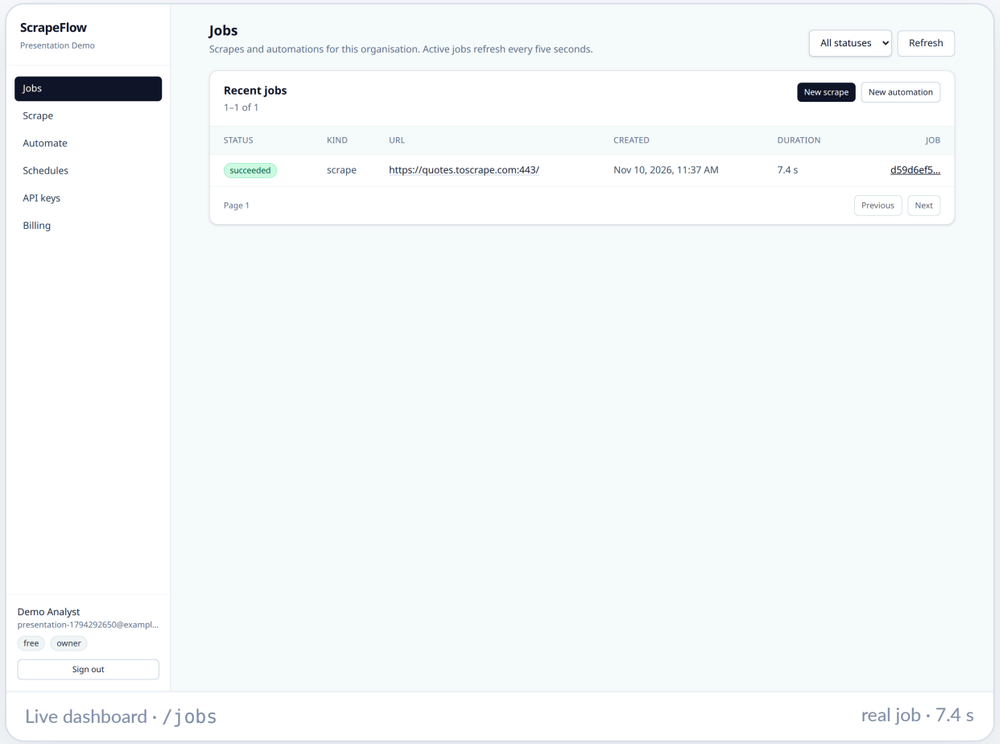
One control plane. One queue. One dashboard.

ScrapeFlow accepts a job over HTTP, returns `202 Accepted` immediately, and does the slow work in isolated workers.

What a user gets

- **Scrape API** — URL, CSS/XPath selectors, page text, links, meta, screenshots, HTML capture.
- **Browser automation** — a typed step DSL: goto, click, type, wait, extract, screenshot.
- **Schedules** — cron expressions, run history, manual trigger.
- **Artifacts** — presigned downloads for screenshot, HTML and JSON, with SHA-256 and expiry.
- **Webhooks** — signed deliveries with a durable outbox and retries.
- **Billing** — Stripe checkout/portal plus a credit ledger and plan quotas.

202 async jobs | SSRF-safe egress | JWT + hashed API keys | Prometheus metrics



Live dashboard · `/jobs` — real job · 7.4 s

4
PLANS: FREE → BUSINESS

15
SQLALCHEMY MODELS

29
HTTP ROUTES IN OPENAPI

CAPABILITY MAP

Everything the platform ships

Each box is implemented in the repository and mapped to a module, not a roadmap promise.

INGEST

Scrape jobs

HTTP fetch or real browser. Named selectors, extraction limits, redirects, timeout, idempotency key.

APPS/API/ROUTERS/SCRAPE.PY

AUTOMATE

Step workflows

Typed actions with limits per plan, browser-only execution, optional screenshots per step.

APPS/WORKER/AUTOMATE.PY

OPERATE

Durable jobs

Queued → running → terminal states, events, cancellation, retries, hard timeout + reaper.

APPS/WORKER/TASKS/JOBS.PY

STORE

Artifacts

S3-compatible upload, SHA-256, content type, retention by plan, presigned download URLs.

PACKAGES/CORE/ARTIFACTS.PY

REPEAT

Schedules

Cron validation via croniter, per-run history, Celery Beat dispatch, manual run endpoint.

APPS/WORKER/TASKS/SCHEDULES.PY

NOTIFY

Signed webhooks

HMAC signatures, encrypted endpoint secrets, outbox delivery and retry tracking.

APPS/WORKER/WEBHOOKS.PY

GOVERN

Tenancy & quota

Organisations, memberships, roles, scoped API keys, credit ledger, concurrency limits.

PACKAGES/CORE/QUOTA.PY

OBSERVE

Ops surface

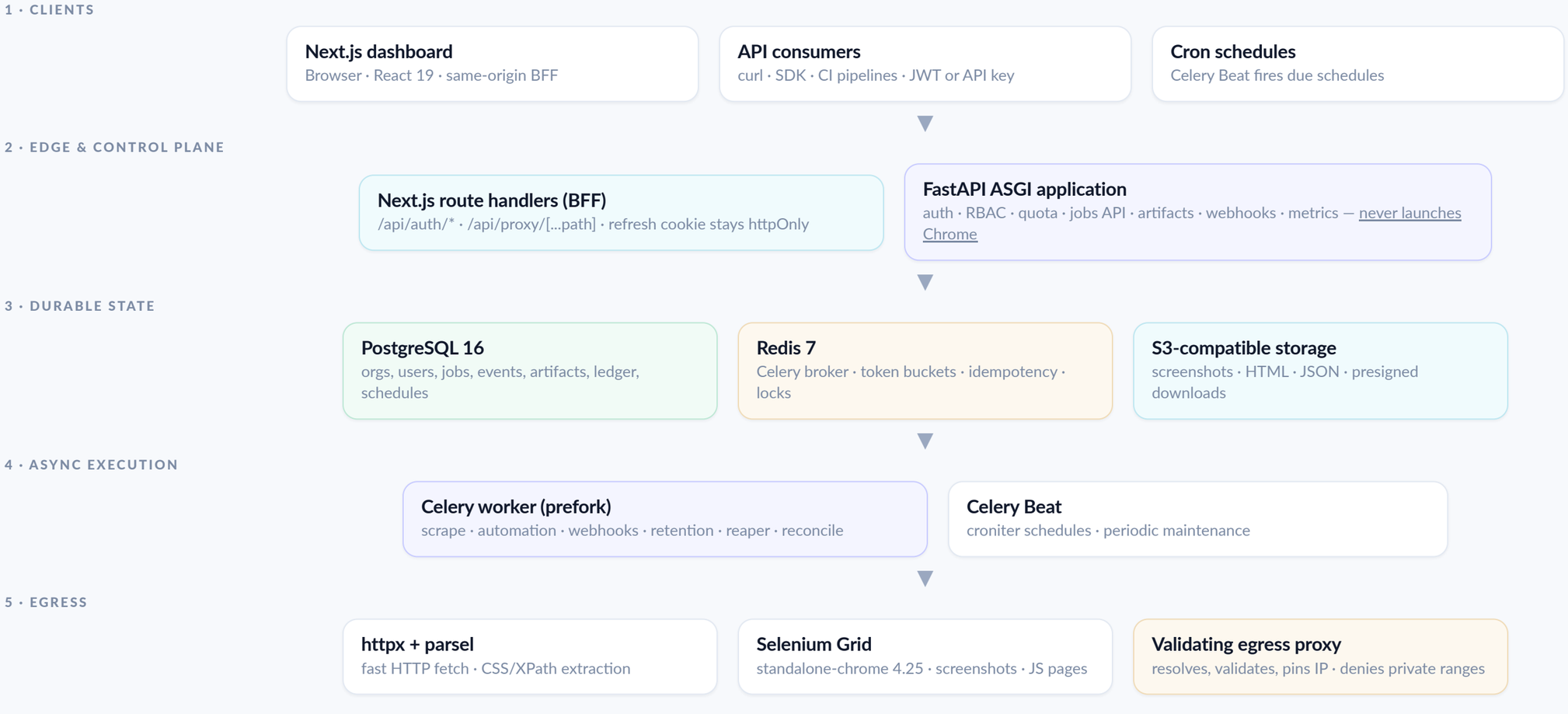
JSON logs with request/job IDs, Prometheus metrics, Sentry, `/health` and `/ready`.

PACKAGES/CORE/OBSERVABILITY.PY

IN SHORT Every capability on this slide maps to a module path, so a claim can be checked against code in one step.

ARCHITECTURE

Control plane and execution plane are deliberately separate



Why split: an HTTP request can return in milliseconds while a 60-second browser job runs elsewhere. API restarts never kill running scrapes.

Why proxy-only egress: DNS is resolved and checked at connect time, then the validated IP is pinned — closing the rebinding window.

Production: the same topology sits behind Caddy 2.8 with automatic TLS in `deploy/compose.prod.yml`.

END-TO-END FLOW

What happens when a user submits a scrape

The real job used throughout this deck: `POST /v1/scrape` → `quotes.toscrape.com`, `render_js=true`, screenshot + HTML.

1 Authenticate & authorize
JWT bearer or hashed API key → membership → role → required scope (`scrape:write`). Audit log written.

2 Validate the payload
Pydantic v2 schema: absolute http(s) URL, selector map, timeout bounds, extraction limits, plan caps.

3 SSRF pre-check & quota reserve
URL validated against the SSRF policy; credits reserved in the usage ledger; rate-limit bucket consumed atomically in Redis.

4 Persist and enqueue
Job row committed as QUEUED ; Celery task published; API returns 202 with the job id and a dedup flag.

5 Execute in the worker
httpx fetch or Selenium Grid browser run through the validating egress proxy. Hard timeout is enforced.

6 Extract
parsel applies CSS/XPath selectors, page text, links and meta limits; the payload is normalised to JSON.

7 Store artifacts
screenshot.png, page.html and result.json uploaded through boto3, hashed with SHA-256 and given a retention expiry.

8 Finalise
Job marked SUCCEEDED with duration; events appended; webhook delivery queued in the outbox.

9 Consume
The dashboard polls `/v1/jobs/{id}` every 5 s; artifacts download through short-lived presigned URLs.

observed timeline

```
$ curl -X POST /v1/scrape ...
202 {"job":{"status":"queued", ...}}

poll 1  succeeded   duration_ms: 7406
result.title       "Quotes to Scrape"
artifacts          result.json · page.html · screenshot.png
```

IN SHORT The API only validates, records and enqueues. Everything slow and failure-prone happens outside the request.

STACK AT A GLANCE

Every layer, language and tool

LAYER	LANGUAGE	TECHNOLOGY	VERSION	ROLE
API control plane	Python	FastAPI · Starlette · Uvicorn	`0.162 · 1.10 · 0.32`	ASGI HTTP API, DI, OpenAPI, async jobs
Validation & config	Python	Pydantic v2 · pydantic-settings	`2.9+ · 2.5+`	Typed request/response models, env settings
Persistence	SQL	SQLAlchemy 2 async · asyncpg · Alembic · PostgreSQL	`2.0 · 0.30 · 1.14 · 16`	ORM, async driver, migrations, relational truth
Job queue	Python	Celery[redis] · redis-py · croniter	`5.4 · 5.2 · 3.0`	Durable workers, broker, cron schedules
Scraping engine	Python	httpx · parsel · Selenium	`0.27 · 1.9 · 4.25`	HTTP fetch, CSS/XPath, real browser
Artifacts	Python	boto3 · S3 API (MinIO / s3mock)	`1.35+`	Uploads, SHA-256, presigned downloads
Security	Python	PyJWT · argon2-cffi · cryptography	`2.9 · 23.1 · 50`	Tokens, password hashing, secret encryption
Observability	Python	structlog · prometheus-client · sentry-sdk	`24.4 · 0.20 · 2.14`	JSON logs, metrics, error reporting
Dashboard	TypeScript	Next.js · React · Tailwind CSS	`16.4.5 · 19.3.1 · 4.3.4`	App Router UI, BFF, styling
Frontend tooling	TypeScript / JS	ESLint · Vitest · Playwright · Vite	`9.39 · 5.1 · 1.65 · 8.3`	Lint, unit tests, E2E, test runner core
Runtime & infra	YAML / HCL-free	Docker Compose v2 · Caddy · Bash · PowerShell	`v2 · 2.8 · 3.2+ / 5.1+`	Local + prod topology, TLS, one-command runners

IN SHORT One row per layer. If a library is not on this slide, it is not in the runtime dependency list.

LANGUAGES

Python for the engine, TypeScript for the cockpit

Deliberate split: heavy I/O and scraping live in Python; the UI and BFF share one typed language.

BACKEND 14,098 LOC · 95 FILES

Python 3.12

Pinned to `>=3.12,<3.13` in `pyproject.toml`.

- **Async first:** native `async/await` for FastAPI, asyncpg and httpx.
- **Ecosystem:** Selenium, parsel, boto3, Celery are mature here.
- **Typing:** full annotations checked by mypy in strict-ish mode.
- **Performance:** I/O-bound work, so the GIL is not the bottleneck.

FRONTEND 5,477 LOC · TS/TSX

TypeScript 5.9

Strict config, `noEmit` typecheck in CI.

- **Contracts:** job, artifact, schedule and webhook types mirror the API.
- **Safety:** discriminated unions for job status and automation steps.
- **Ergonomics:** same language in React components and server route handlers.
- **Compile-time only:** no runtime overhead in the browser.

SUPPORTING GLUE & INFRASTRUCTURE

SQL · YAML · Bash · PowerShell

- **SQL:** Alembic migrations and PostgreSQL DDL; indexes and constraints do real work.
- **YAML:** Compose dev/prod topology, CI workflow, alert rules.
- **Bash 3.2+:** `run.sh` + `run-macos.sh` for Linux/macOS.
- **PowerShell 5.1+:** `run.ps1` + `run.cmd` for native Windows.

3.12
PYTHON, PINNED MINOR

5.9
TYPESCRIPT

2
LANGUAGES IN APP CODE

100%
TYPE-ANNOTATED APP CODE

IN SHORT Two application languages, one rule: typed at the boundary, tested at the edge.

PACKAGE MANAGERS

uv on the backend, npm on the frontend

Both lock the full dependency graph. Neither relies on a globally installed toolchain.

uv — Python

backend

- **One tool:** resolves, installs, locks and runs — replaces pip, pip-tools, virtualenv and tox.
- **Reproducible:** `uv.lock` pins every transitive package with hashes.
- **Version pin:** `requires-python = ">=3.12,<3.13"`; `.python-version` keeps machines aligned.
- **Dependency groups:** the app, `dev` (pytest, ruff, mypy, respx, pip-audit) and `e2e` (locust) install independently.
- **Same in Docker:** images use `uv sync --frozen`, so local and container builds match.

```
# everyday commands
uv sync                    # install exactly the lockfile
uv run pytest -q           # run inside the project env
uv run ruff check apps packages tests
uv add "httpx"             # update pyproject + lock
```

npm — Node.js

frontend

- **Bundled with Node 22:** no extra version manager needed to start.
- **Lockfile:** `package-lock.json` pins the tree; `npm ci` is used for clean installs.
- **Scripts as interface:** `dev`, `build`, `start`, `lint`, `typecheck`, `test`, `e2e`.
- **Exact versions:** no ^ ranges for the app runtime — Next, React and tooling are pinned.
- **Docker:** the web image builds with the same lockfile and runs `node server.js`.

```
# scripts in apps/web/package.json
npm ci
npm run lint
npm run typecheck
npm test        # vitest run
npm run build   # next build
```

PYTHON LOCK	NODE LOCK	SERVICE IMAGES	BROWSER IMAGE
`uv.lock`	`package-lock.json`	`postgres:16 · redis:7`	`standalone-chrome:4.25`

IN SHORT Lockfiles are committed. A fresh clone resolves to the exact versions that passed the gates.

BACKEND LIBRARIES

The Python stack, group by group

Declared in `scrapeflow/pyproject.toml`; every group below is exercised by the test suite.

HTTP / API

Web layer

`fastapi`
`starlette`
`uvicorn[standard]`

Routing, DI, ASGI serving, request lifecycle.

VALIDATION

Schemas

`pydantic>=2.9`
`pydantic-settings`
`email-validator`

Request/response models, env config, email types.

DATA

Persistence

`sqlalchemy[asyncio]`
`asyncpg`
`alembic`

ORM, async PostgreSQL driver, versioned migrations.

QUEUE

Async work

`celery[redis]`
`redis>=5.2`
`croniter`

Workers, broker/client, cron expression evaluation.

SCRAPING

Fetch & parse

`httpx`
`parsel`
`selenium`

HTTP client, CSS/XPath engine, WebDriver protocol.

STORAGE

Artifacts

`boto3>=1.35`

S3 uploads, metadata, presigned URL generation.

SECURITY

Identity

`PyJWT`
`argon2-cffi`
`cryptography`

Signed tokens, password hashing, encrypted secrets.

OPS

Observability

`structlog`
`prometheus-client`
`sentry-sdk[fastapi]`

Structured logs, metrics registry, exception capture.

Billing: `stripe>=11,<15` for checkout sessions, the customer portal and signed webhook events.

Test-only: pytest 9, pytest-asyncio, pytest-cov, respx, ruff, mypy, pip-audit, locust (e2e group).

Guardrail: `pip-audit` scans the resolved lock for known CVEs; mypy checks 94 source files.

IN SHORT Eight groups, one responsibility each — no library is present for decoration.

BACKEND · WEB LAYER

FastAPI, Starlette, Uvicorn, Pydantic

Four libraries with four distinct jobs. Each one is replaceable in theory and load-bearing in practice.

FastAPI `0.162`

the API surface

- **Routers per resource:** auth, scrape, automate, jobs, keys, schedules, webhooks, billing, exports, metrics, health.
- **Dependency injection:** `Depends()` resolves the DB session, current principal, org and required scope.
- **Async endpoints:** all I/O is awaited; long work is offloaded, never awaited in-request.
- **Contract generation:** `response_model` produces the OpenAPI schema served at `/docs`.

Starlette `1.10`

the foundation

ASGI toolkit underneath FastAPI. Used directly for the `RequestContextMiddleware` that accepts a safe `X-Request-ID`, binds it to logs, and rejects unsafe values.

Uvicorn `0.32 [standard]`

the server

Runs the ASGI app on port 8000 inside the container. `[standard]` adds uvloop and httptools for throughput; health probes hit `/health` and `/ready`.

Pydantic v2 `2.9+`

the contract

Rust-backed validation and serialisation. `pydantic-settings` loads `.env` into a typed `Settings` object at startup, so a bad secret fails fast.

```python
# apps/api/routers/scrape.py — shape (simplified)
@router.post(
    "",
    response_model=JobAccepted,
    status_code=status.HTTP_202_ACCEPTED,
)
async def create_scrape(
    payload: ScrapeRequest,
    principal: Principal = Depends(require_scope("scrape:write")),
    session: AsyncSession = Depends(get_session),
    idempotency_key: str | None = Header(None, alias="Idempotency-Key"),
) -> JobAccepted:
    # validate → quota → persist queued job → enqueue
    job = await dispatch.create_job(...)
    return JobAccepted(job=job, deduplicated=False)
```

WHY THIS COMBINATION

- **OpenAPI for free:** 29 documented routes become a browsable, testable contract.
- **One validation layer:** the same Pydantic models guard HTTP input and serialise output.
- **Typed settings:** missing DATABASE_URL is a startup error, not a 3 a.m. surprise.

BACKEND · PERSISTENCE

SQLAlchemy async, asyncpg, Alembic, PostgreSQL 16

PostgreSQL is the source of truth. Redis is a cache and a queue — never the record of what happened.

SQLAlchemy 2.0 (async)

`AsyncSession` per request, declarative models, typed `select()`. Relationships and constraints live in the model layer, not in ad-hoc SQL.

asyncpg 0.30

The actual PostgreSQL driver. Binary protocol, prepared statements, no psycopg overhead. URL form: `postgresql+asyncpg://…`.

Alembic 1.14

Five versioned migrations run by the `migrate` compose service before the API starts. `alembic current` / `heads` are documented runbook commands.

Pooling fix that matters

Celery tasks create and destroy an event loop per job. Reusing a pooled asyncpg connection across loops caused `Future attached to a different loop`; workers now use `NullPool` and a regression test runs the reaper across two loops.

15 MAPPED TABLES

organizations	users
memberships	api_keys
refresh_tokens	audit_logs
jobs	job_events
artifacts	schedules
schedule_runs	webhook_endpoints
webhook_deliveries	usage_ledger
stripe_events	+ enums & indexes

STATE MACHINE STORED ON JOBS

```
queued → running → succeeded
              ↘ failed · timed_out · cancelled
```

IN SHORT PostgreSQL is the record of what happened. Redis may lose data without rewriting history.

BACKEND · ASYNC WORK

Celery, Redis, Beat and croniter

The API answers in milliseconds; workers do the slow, failure-prone part with retries and a hard deadline.

Celery 5.4

worker

- **Prefork pool:** isolation from crashes and blocking Selenium calls.
- **Retries:** `max_attempts=2` recorded on the job row.
- **Task families:** jobs, reaper, reconcile, retention, schedules, webhooks, health.
- **Multiprocess metrics:** prefork counters aggregated through `PROMETHEUS_MULTIPROC_DIR` on port 9101.

Redis 7

broker + state

- **Broker:** Celery task transport with append-only persistence enabled.
- **Token bucket:** atomic Lua script per key/org — no race between read and write.
- **Idempotency:** `Idempotency-Key` replays the original job instead of double-charging.
- **Locks & counters:** short-lived coordination that is safe to lose.

Beat + croniter

time

- **Beat scheduler:** separate container, schedule file on a tmpfs.
- **croniter 3.0:** validates cron expressions and computes the next fire time.
- **Run history:** every fire creates a `schedule_runs` row with a terminal status.
- **Manual trigger:** `POST /v1/schedules/{id}/run` for testing.

worker + beat (live stack)

```
$ docker compose ps --format '{{.Service}}  {{.Status}}'
api           Up (healthy)
worker        Up
beat          Up
redis         Up (healthy)
postgres      Up (healthy)

# Celery Beat schedules due work every minute;
# the reaper closes orphaned browser sessions and hard-times-out jobs.
[tasks] jobs · reaper · reconcile · retention · schedules · webhooks
```

Failure handling, explicitly

- **Hard timeout:** a job cannot run forever; it moves to `timed_out`.
- **Reaper:** scheduled task kills orphan browser sessions and reconciles stuck rows.
- **Reconcile:** compares broker and database state after a crash.
- **Retention:** expires artifacts according to the plan's retention days.
- **Webhook outbox:** deliveries survive a worker restart and retry with backoff.

BACKEND · SCRAPING ENGINE

httpx for speed, Selenium for truth, parsel for shape

Two execution paths behind one contract, both forced through the same validating egress proxy.

httpx 0.27

fast path

- Async HTTP client with explicit timeouts and redirect limits.
- Used when `render_js=false`; typically a fraction of a browser run.
- Custom headers, user agent, proxy routing via the egress proxy.
- Response status, final URL and redirect chain are recorded in the result.

Selenium 4.25

browser path

- Remote WebDriver against `standalone-chrome:4.25.0` in Selenium Grid.
- Waits for a selector, captures screenshots and HTML.
- Powers the automation step DSL: goto, click, type, wait, extract, screenshot.
- Sessions are bounded and reaped; the API container never launches Chrome.

parsel 1.9

extraction

- CSS and XPath selectors over the fetched HTML.
- Attribute extraction, multi-match lists, named result fields.
- Page text, links and meta tags with hard limits.
- Result is normalised to plain JSON for storage and export.

SSRF DEFENCE IN DEPTH

- **Policy layer:** scheme allow-list, port rules, private/link-local/loopback denial at request time.
- **Validating proxy:** resolves DNS, checks every address, then connects to the pinned IP — not the hostname.
- **Chained upstreams:** optional commercial proxies sit *behind* the validator, never around it.

real selector result

```
$ jq '.selectors' result.json
{
  "author": "Albert Einstein",
  "first_quote": "“The world as we have created it is a
   process of our thinking. It cannot be changed
   without changing our thinking.”"
}
```

IN SHORT Two engines, one contract: the caller gets the same result shape whether or not JavaScript was required.

BACKEND · OBJECT STORAGE

Artifacts: boto3 over an S3-compatible API

Screenshots and HTML are too large for a database row and too useful to lose. They live in object storage, addressed by row metadata.

boto3 1.35+

Single S3 client configured by `ARTIFACTS_ENDPOINT`, bucket and credentials. Works against MinIO, AWS S3, or `adobe/s3mock` locally.

Write path

Worker uploads `screenshot.png`, `page.html` and `result.json`, computes SHA-256, then inserts an `artifacts` row with content type, size and `expires_at`.

Read path

`GET /v1/artifacts/{id}/download` authorises the caller, then redirects to a short-lived presigned URL. The browser never sees storage credentials.

Retention

Plan-dependent: 1 day on Free, 7 Starter, 30 Pro, 90 Business. The retention task deletes expired objects and rows.

REAL ARTIFACTS FROM JOB D59D6EF5…

KIND	FILE	SIZE	SHA-256 (PREFIX)
result	`result.json`	`2.1 KB`	`27906a08a7cf…`
html	`page.html`	`11 KB`	`47c8812e633f…`
screenshot	`screenshot.png`	`84 KB`	`54ebbeb9b150…`

DOWNLOAD_URL RETURNED BY THE API

```
/v1/artifacts/2b6284ac-…/download
→ 307 redirect → presigned S3 URL
```

Local note: the pinned MinIO tag returned `401` from its registry in this environment, so the verified run used `adobe/s3mock`. The S3 code path and presigned redirects are identical; the official MinIO image/lifecycle remains unverified.

IN SHORT The database stores metadata, object storage stores bytes, and the API stores neither.

BACKEND · SECURITY

Identity, secrets and egress are separate defences

AUTHENTICATION

PyJWT 2.9

Short-lived access tokens signed HS256; claims carry user, org and role. Refresh tokens rotate and are stored hashed.

PASSWORDS

argon2-cffi

Argon2id hashing with per-password salt. No plaintext, no reversible storage, constant-time verification.

API KEYS

Pepper + SHA-256

Keys are shown once. Only a hash with a server-side pepper is stored, so a database leak alone cannot replay them.

SECRETS AT REST

cryptography 50

Webhook signing secrets and proxy credentials are encrypted with Fernet before they reach PostgreSQL.

Authorisation

- **Roles:** owner, admin, member.
- **Scopes:** `scrape:write`, `automation:write`, `jobs:read`, `schedules:write`, `exports:read`, `keys:manage`, `billing:manage`, `audit:read`.
- **Enforcement:** a dependency resolves the required scope before the handler runs.
- **Audit:** sensitive actions append to `audit_logs`.

Session model

- Access token kept **in memory** in the dashboard.
- Refresh token in an **httpOnly, same-site cookie**.
- Rotation on refresh; logout revokes the stored token.
- No tokens in `localStorage`; the BFF keeps the browser same-origin.

Egress control

- Private, loopback, link-local and metadata ranges denied.
- DNS resolved and validated at connect time.
- Validated IP pinned to close rebinding.
- Header and byte caps; hop-by-hop headers stripped.

PII and logging: Sentry initialises with `send_default_pii=False`; structured logs carry identifiers, not secrets. `/metrics` can be disabled or protected with `METRICS_TOKEN`.

IN SHORT Compromise one layer and the next still holds: hashed credentials, scoped tokens, validated egress.

BACKEND · COMMERCIAL LAYER

Tenancy, quotas, plans and Stripe

Isolation is structural: every query is scoped by organisation, and every job is priced before it runs.

Organisation scope

`organizations` own users through `memberships`, and own jobs, keys, schedules, webhooks, artifacts and ledger rows.

Plan catalog is code

Plans are frozen dataclasses in `packages/core/plans.py` — not editable database rows. A billing change is a reviewed code change with tests.

Credit weights

base 1 · browser +1 · screenshot +1 · proxy 2 · each extra page +1 · each started 10 automation steps +1.

Usage ledger

Reserve → settle → release. A failed job does not silently consume credits; balances are reconstructable from rows.

PLAN	CREDITS / MO	CONCURRENCY	STEPS	PAGES	RETENTION
Free	100	1	10	5	1 day
Starter	5,000	3	50	25	7 days
Pro	50,000	10	100	100	30 days
Business	250,000	25	200	500	90 days

Stripe `stripe 11–15`

billing

- **Checkout:** `POST /v1/billing/checkout` creates a subscription session.
- **Portal:** `POST /v1/billing/portal` hands plan changes and invoices to Stripe.
- **Webhook:** `POST /v1/billing/webhook` verifies the signature and records the event id in `stripe_events` for idempotency.
- **Plans endpoint:** `GET /v1/billing/plans` and `/v1/billing/usage` drive the dashboard.

VERIFICATION STATUS

The integration code and webhook idempotency are implemented and unit-tested. **Live Stripe checkout was not executed** — no API keys or test account were available. It is listed as an unverified external path, not as working revenue.

RATE LIMITING

Atomic Redis token bucket with Lua: capacity, refill rate and cost are arguments, and the script returns allowed / remaining / retry-after / reset. Response headers make throttling visible to clients.

IN SHORT Pricing lives in code, usage lives in rows, and limits are enforced before work starts.

BACKEND · OBSERVABILITY

Logs you can join, metrics you can alert on

Phase 9 added correlation, metrics and error reporting; the evidence below is from the running stack, not a mock.

structlog 24.4

- JSON logs with `request_id` on every API line.
- Worker logs bind `job_id` for the whole task.
- `X-Request-ID` accepted only when safe, generated otherwise.

prometheus-client 0.20

- HTTP counters/histograms by route template — no cardinality explosion.
- Queue depth, browser sessions, webhook backlog, DB and Redis latency.
- `/metrics` excluded from its own counters; worker prefork aggregation on 9101.

sentry-sdk 2.14

- FastAPI integration for unhandled exceptions.
- `send_default_pii=False` by design.
- Optional DSN: absent means disabled, not crashing.

GET /metrics · live

```
$ curl -s 127.0.0.1:8000/metrics | grep '^scrapeflow_'
# method label trimmed to fit the slide
scrapeflow_http_requests_total{route="/health",status="200"} 82.0
scrapeflow_http_requests_total{route="/v1/scrape",status="202"} 1.0
scrapeflow_http_requests_total{route="/v1/jobs/{job_id}",status="200"} 3.0
scrapeflow_http_requests_total{route="/v1/artifacts/{id}/download",status="307"} 5.0

$ curl -s 127.0.0.1:8000/ready
{"status":"ready","checks":{"postgres":{"status":"ok"},"redis":{"status":"ok"}}}
```

PROBES

`/health` is liveness (process up). `/ready` checks PostgreSQL and Redis and returns `503` when either fails — that is what Compose and Caddy should gate on.

ALERTING ASSETS

Prometheus scrape config, alert rules and a Grafana dashboard are committed and configuration-validated. Prometheus/Grafana themselves were not started in this environment.

CAPACITY HONESTY

A Locust profile exists and its shape is unit-tested, but **Locust is not installed here**. No throughput number is claimed; the 3 skipped tests record that gap.

IN SHORT If a job cannot be explained from logs, metrics and events, it is not operable.

FRONTEND LIBRARIES

Next.js 16, React 19, TypeScript, Tailwind 4

A deliberately small dependency surface: three runtime packages, and everything else is tooling.

FRAMEWORK

Next.js 16.4.5

App Router, server route handlers for the BFF, file-based pages, production build with `next build`.

UI RUNTIME

React 19.3.1

Client components for interactive forms and polling; server components for page shells. `react-dom` pinned to the same version.

LANGUAGE

TypeScript 5.9.3

Strict checking via `next typegen && tsc --noEmit`. API types live in `lib/types.ts`.

STYLING

Tailwind CSS 4.3.4

PostCSS plugin, utility classes in components, no runtime CSS-in-JS. Design tokens are Tailwind's palette plus a small set of shared primitives.

TOOLING (DEVDEPENDENCIES)

ESLint	`9.39.5`	flat config + `eslint-config-next`
Vitest	`5.1.0`	unit tests for pure UI logic
Vite	`8.3.6`	transform engine behind Vitest
Playwright	`1.65.0`	browser E2E + PDF rendering
TypeScript	`5.9.3`	typecheck
PostCSS plugin	`4.3.4`	Tailwind pipeline

RUNTIME DEPENDENCIES

`next 16.4.5` `react 19.3.1` `react-dom 19.3.1`

No state library, no fetch wrapper library, no component kit. Auth state is a React context; HTTP is a small typed `request()` helper; UI primitives are local.

3 RUNTIME PACKAGES

9 TOP-LEVEL ROUTES

30 VITEST TESTS

7 PLAYWRIGHT SPECS RUN

IN SHORT Three runtime packages. Every other name on this slide is a build-time tool.

The dashboard talks to a same-origin BFF, not to the API

One origin in the browser solves CORS, cookie policy and token exposure at the same time.

SERVER ROUTE HANDLERS

`/api/auth/signup`	creates org + owner, sets refresh cookie
`/api/auth/login`	credentials → access token + cookie
`/api/auth/refresh`	rotates refresh token, returns access token
`/api/auth/logout`	revokes and clears the cookie
`/api/proxy/[...path]`	authorised pass-through to FastAPI

Browser routes

/login · /signup · /jobs · /jobs/[id] · /scrape · /automate · /schedules · /keys · /billing

WHY A BFF

- **Token safety:** the refresh token lives in an httpOnly cookie the JavaScript cannot read.
- **No CORS:** the browser only ever calls its own origin.
- **In-memory access token:** a hard refresh calls `/api/auth/refresh` and restores the session.
- **One place to change:** upstream API URL is server configuration, not a client constant.

```
// conceptual flow
Browser  ──fetch──▶  Next.js /api/proxy/*
                         │  attaches access token
                         ▼
                     FastAPI /v1/*
                         │  JSON + status
                         ▼
Browser  ◀── same-origin response
```

Trade-off: every API call takes one extra hop. In exchange, the app has no CORS configuration, no token in web storage, and a single auditable boundary.

IN SHORT The browser never holds a long-lived credential — and never needs a CORS configuration.

FRONTEND · UI SYSTEM

Tailwind utilities plus a small local component kit

No design-system dependency. Nine primitives and four editors cover every screen in the product.

PRIMITIVES · COMPONENTS/UI.TSX

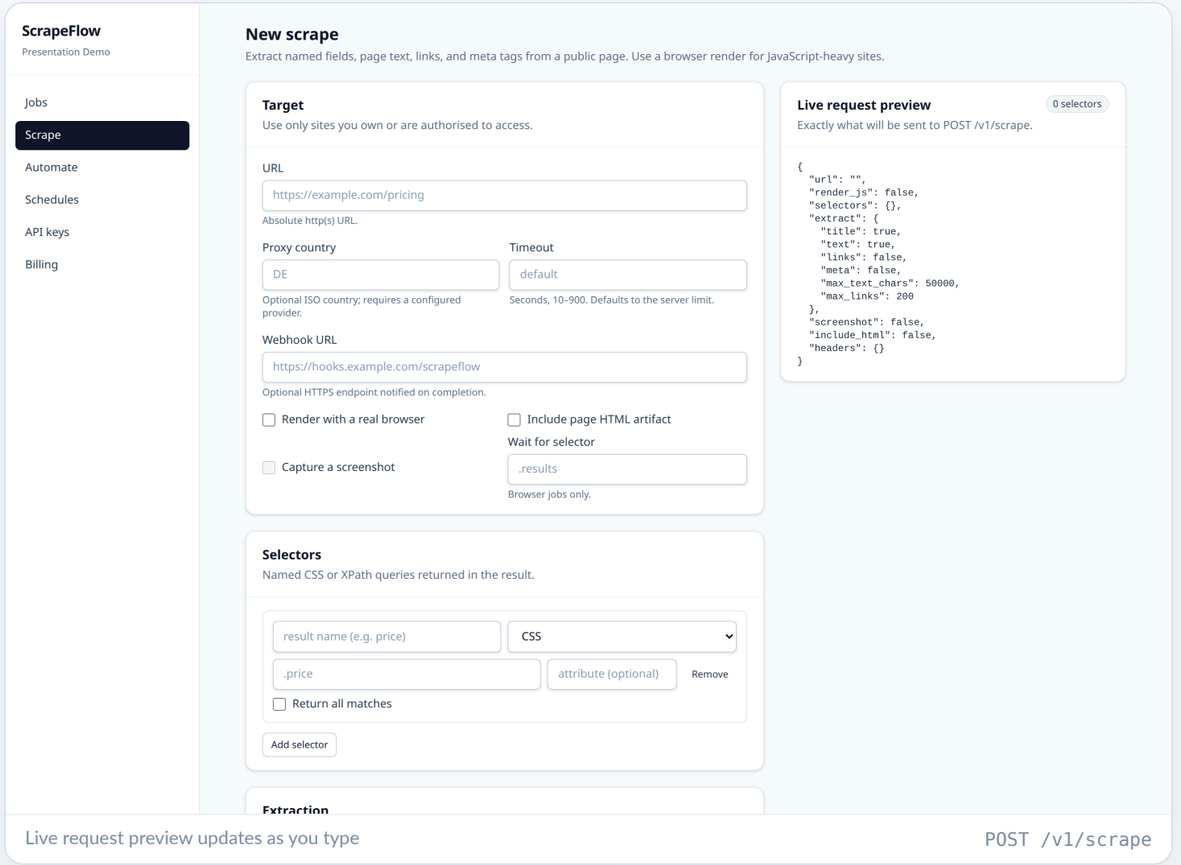
Card · CardHeader · Button · Input · Select · Field · Badge · Alert · Spinner · EmptyState · CopyButton

COMPOSITE EDITORS

- **SelectorEditor:** name + CSS/XPath + optional attribute + multi-match.
- **KeyValueEditor:** arbitrary request headers as rows.
- **ExtractOptionsEditor:** title, text, links, meta and hard limits.
- **Automation step builder:** typed actions with per-action defaults.

FEEDBACK PATTERNS

Inline errors from `ApiError`, optimistic disabled states while submitting, skeletons/spinners for polling pages, and empty states that explain the next action.



STYLING

Tailwind 4 utility classes, no CSS-in-JS runtime, consistent slate/indigo palette.

ACCESSIBILITY

Labels bound to inputs, real buttons, visible focus states, semantic headings.

FRONTEND · QUALITY TOOLING

Lint, typecheck, unit test, build, then E2E

Five gates, all runnable from `apps/web` with npm scripts. The same commands run in CI.

apps/web · verified gates

```
$ npm run lint
✔ eslint .  no errors

$ npm run typecheck
✔ next typegen && tsc --noEmit

$ npm test
✔ 30 tests passed  (3 files)

$ npm run build
✔ compiled successfully
✔ route table generated

$ npm run e2e
✔ 5 passed · 2 skipped  (Chromium)
```

ESLint 9.39

Flat config with `eslint-config-next`; catches React hook mistakes and Next-specific pitfalls.

Vitest 5.1

Pure-logic tests for formatting, safe downloads and automation payload building — fast, no browser needed.

Playwright 1.65

The journey spec drives signup → scrape → artifacts in Chromium. The two skips are environment-gated browser paths.

next build

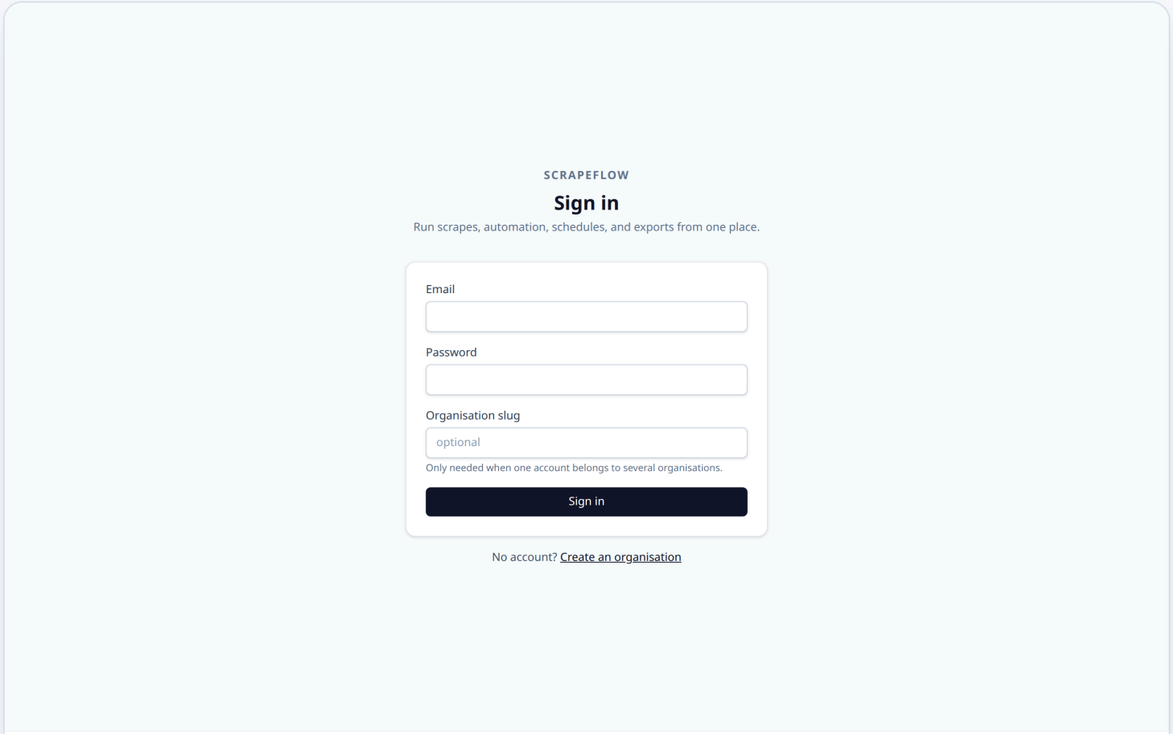
Production compile catches server/client boundary errors that typecheck alone cannot. The web image runs the built output.

IN SHORT Fast unit tests for logic, one real browser journey for the wiring, a production build for the boundary.

LIVE DASHBOARD · ONBOARDING

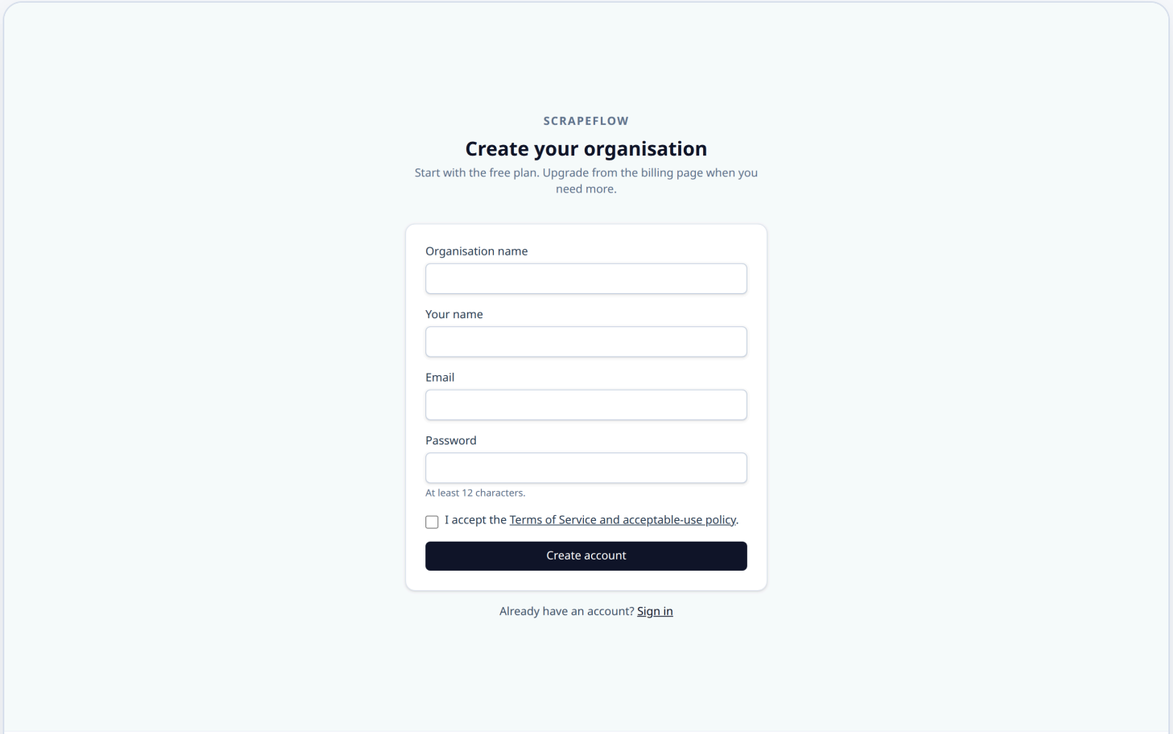
Sign-up creates an organisation; sign-in restores a session

Captured from the running stack at `127.0.0.1:3000`, against real PostgreSQL and Redis.



`/login` — email · password · optional org slug

- **Org slug is optional:** only needed when one identity belongs to more than one organisation.
- **No token in storage:** the refresh cookie is set by the server route handler.



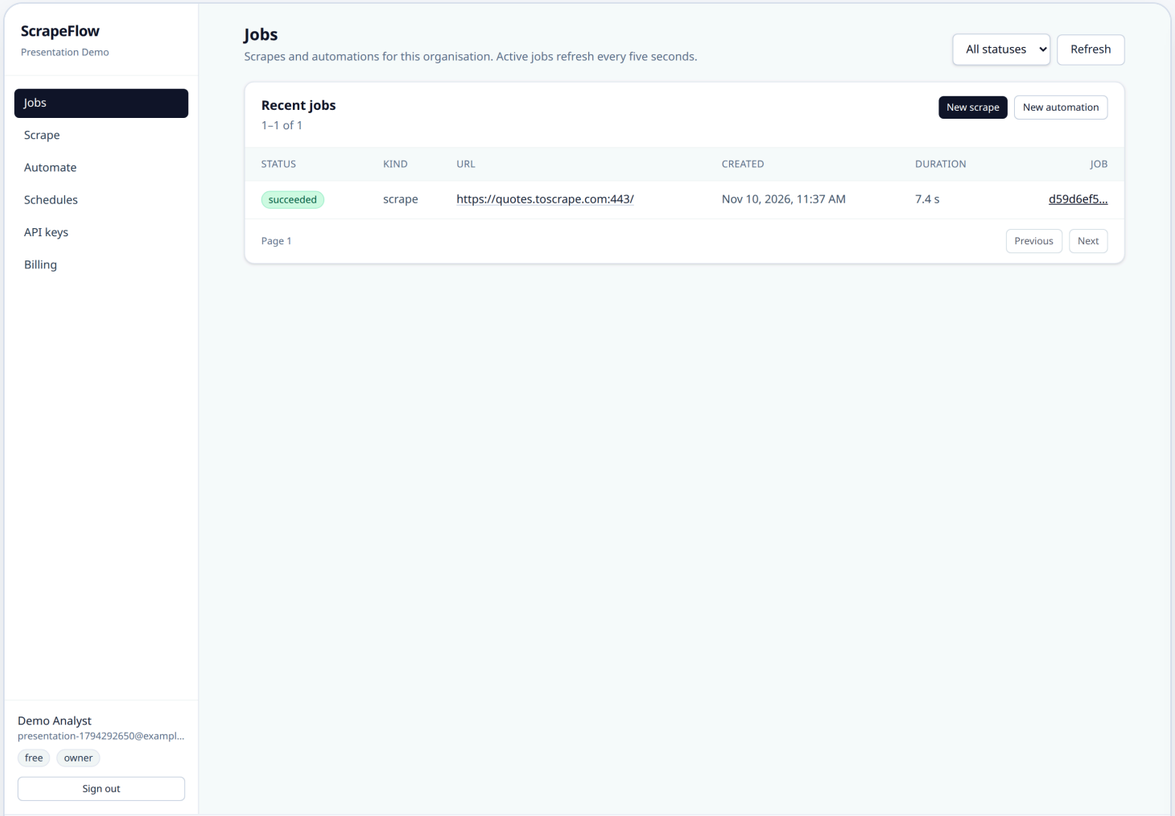
`/signup` — creates org + owner + free plan

- **Tos gate:** the request carries `tos_accepted: true` and the API validates it.
- **Owner role:** the first user gets the owner membership and the free plan with 100 credits.

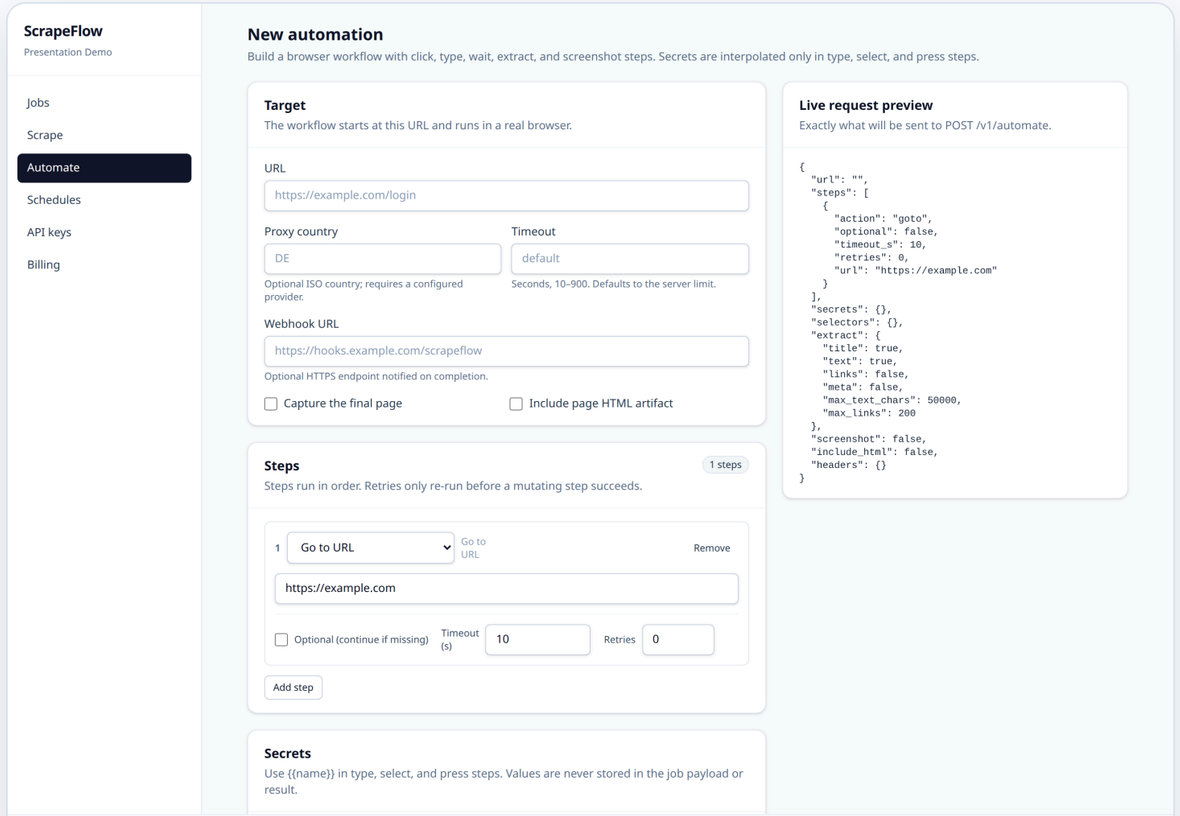
LIVE DASHBOARD · DAILY USE

The job list and the scrape builder

One real succeeded job is visible: `quotes.toscrape.com`, 7.4 seconds, screenshot and HTML requested.



`/jobs` · active jobs refresh every 5 s — 1–1 of 1



`/automate` · typed step builder — browser-only

WHAT THE LIST EXPOSES

Status badge, kind, URL, created time, duration and short job id. Filters by status, pagination, and quick actions to start a new scrape or automation.

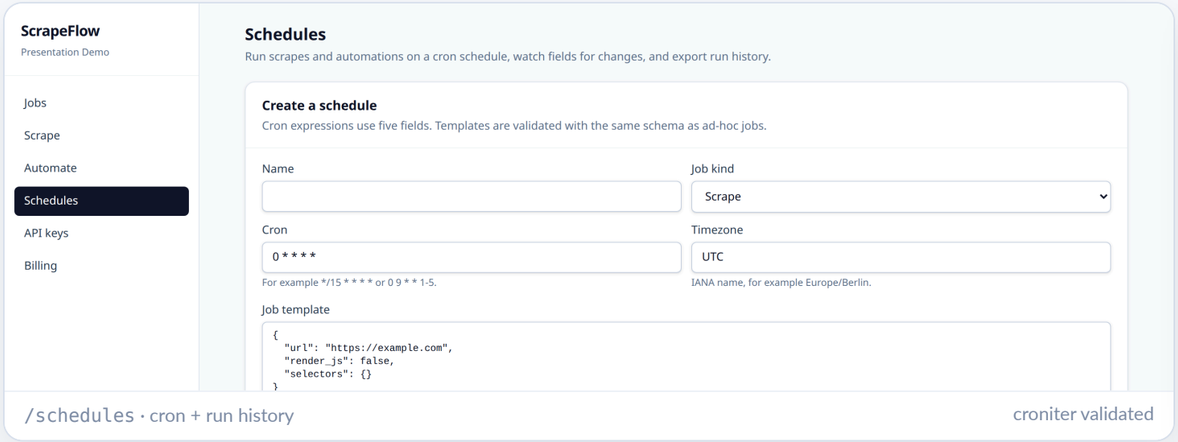
THE SCRAPE BUILDER, BY CONTRAST

The scrape form is declarative: URL, selectors, extraction options, screenshot/HTML flags, proxy country, timeout and webhook — with a live JSON preview of the exact request body.

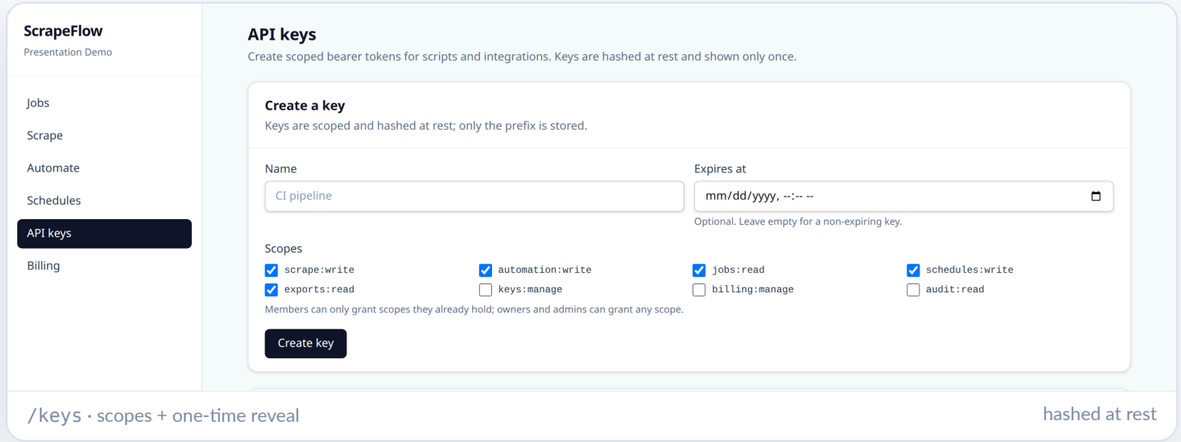
LIVE DASHBOARD · OPERATIONS

Schedules, API keys and billing

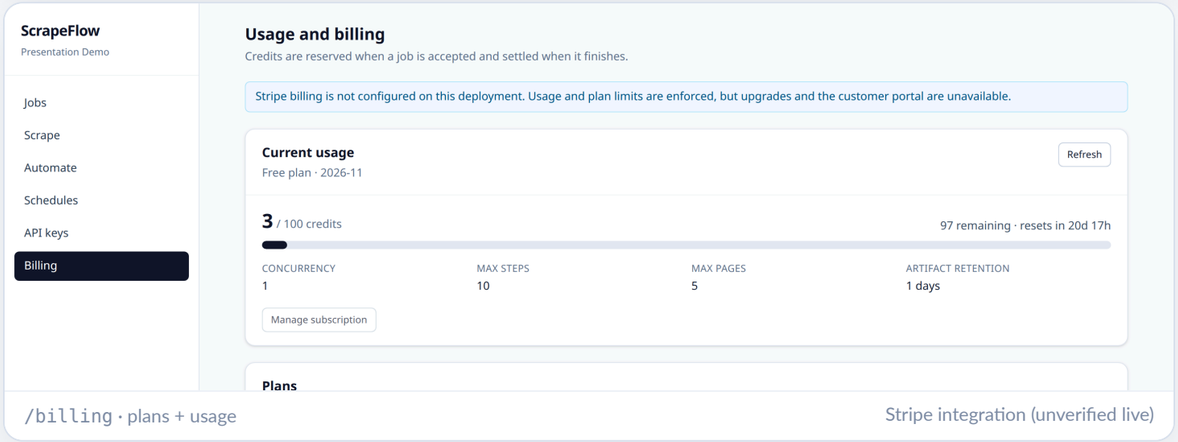
Four more screens, captured from the same session. Each is wired to real endpoints, not static mock data.



/schedules · cron + run history — croniter validated



/keys · scopes + one-time reveal — hashed at rest



/billing · plans + usage — Stripe integration (unverified live)

What each screen proves

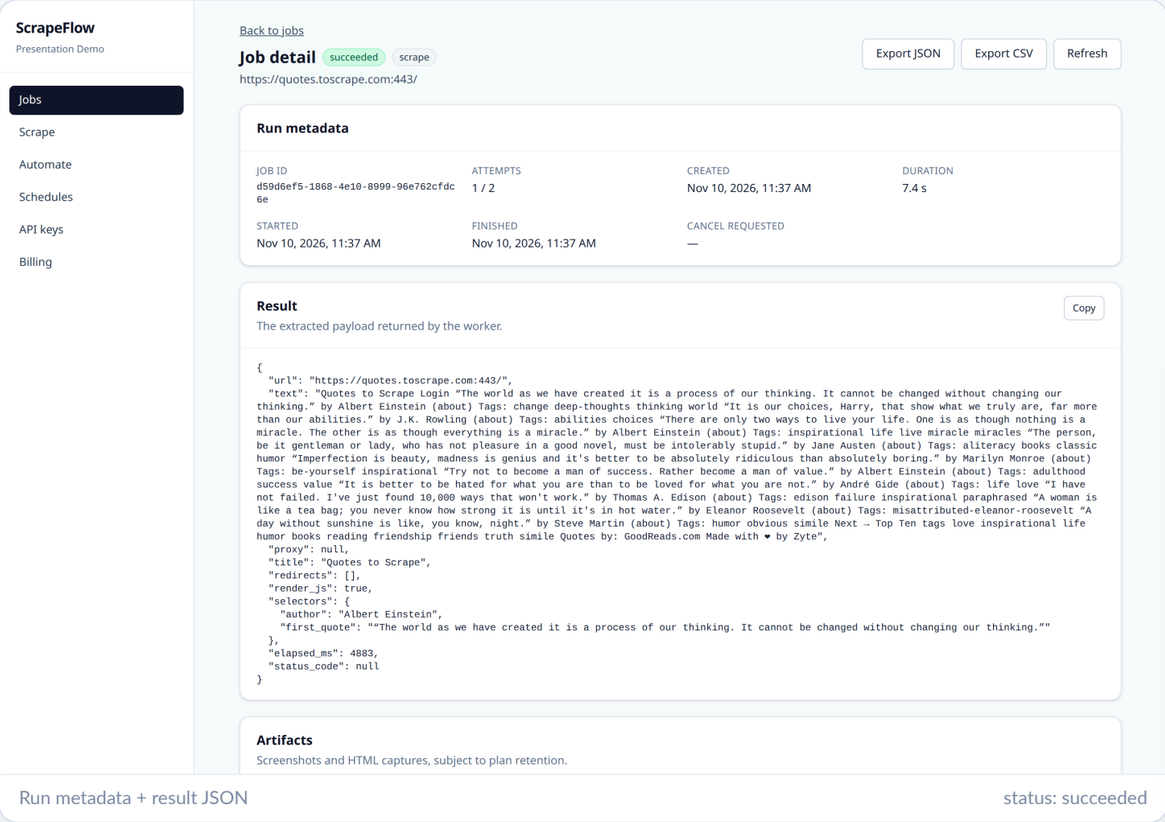
- **Schedules:** cron expressions are validated server-side and every fire is recorded.
- **API keys:** scoped keys are created once and stored hashed; the secret is never shown again.
- **Billing:** plan catalog, credit balance and usage are read from the API; checkout requires live Stripe keys.
- **Consistency:** all four share the same shell, auth context and error handling.

IN SHORT Schedules, keys and billing are first-class screens, not settings hidden in a menu.

LIVE ARTIFACT · JOB DETAIL

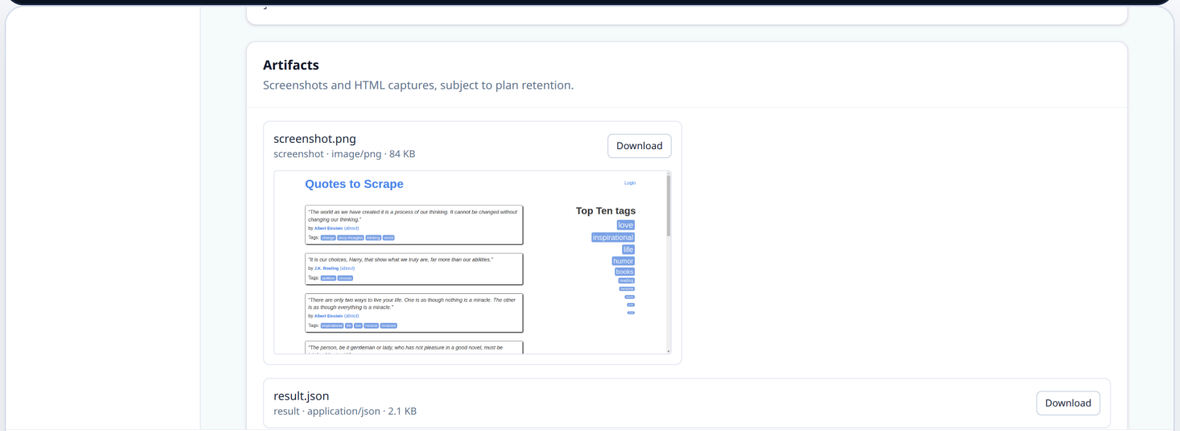
A real job, end to end

Job `d59d6ef5-1868-4e10-8999-96e762cfdc6e` — queued and succeeded in **7.4 s** with attempt 1 of 2.



Run metadata + result JSON — status: succeeded

```
# page.html (excerpt, real artifact · 11 KB)
<html lang="en"><head>
  <meta charset="UTF-8">
  <title>Quotes to Scrape</title>
</head><body>
  <div class="quote" itemscope>
    <span class="text">“The world as we have
      created it is a process of our thinking…”</span>
    <small class="author">Albert Einstein</small>
```



Artifacts card · inline image preview — download via presigned URL

RESULT TITLE
Quotes to Scrape

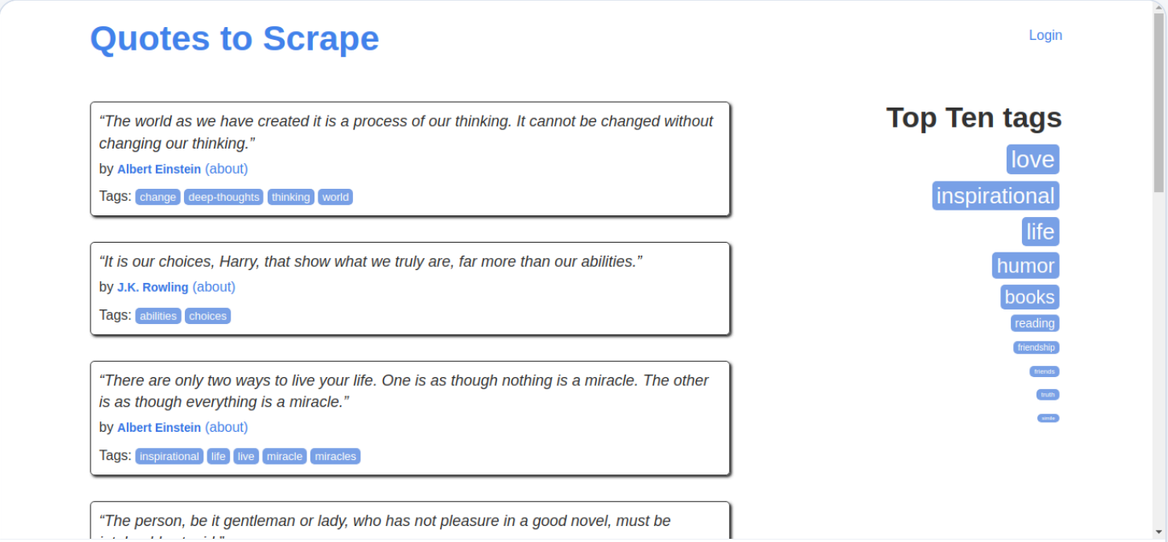
SELECTOR: AUTHOR
Albert Einstein

ARTIFACTS
3 files · screenshot, HTML, JSON

LIVE ARTIFACT · RAW OUTPUT

What the browser actually returned

The screenshot is the page as Chrome rendered it after JavaScript; the JSON is the extracted payload. Both were downloaded from object storage.



screenshot.png · 1366 × 629 · 84 KB — Chrome · Selenium Grid

SHA-256 verified | content_type image/png | expires_at set

```
# result.json (excerpt, real artifact)
{
  "elapsed_ms": 2682,
  "proxy": null,
  "redirects": [],
  "render_js": true,
  "selectors": {
    "author": "Albert Einstein",
    "first_quote": ""“The world as we have created it is a
      process of our thinking. It cannot be changed
      without changing our thinking.”""
  },
  "status_code": null,
  "text": "Quotes to Scrape Login “The world as we
    have created it is a process of our thinking…”"
}
```

THREE ARTIFACTS, THREE JOBS

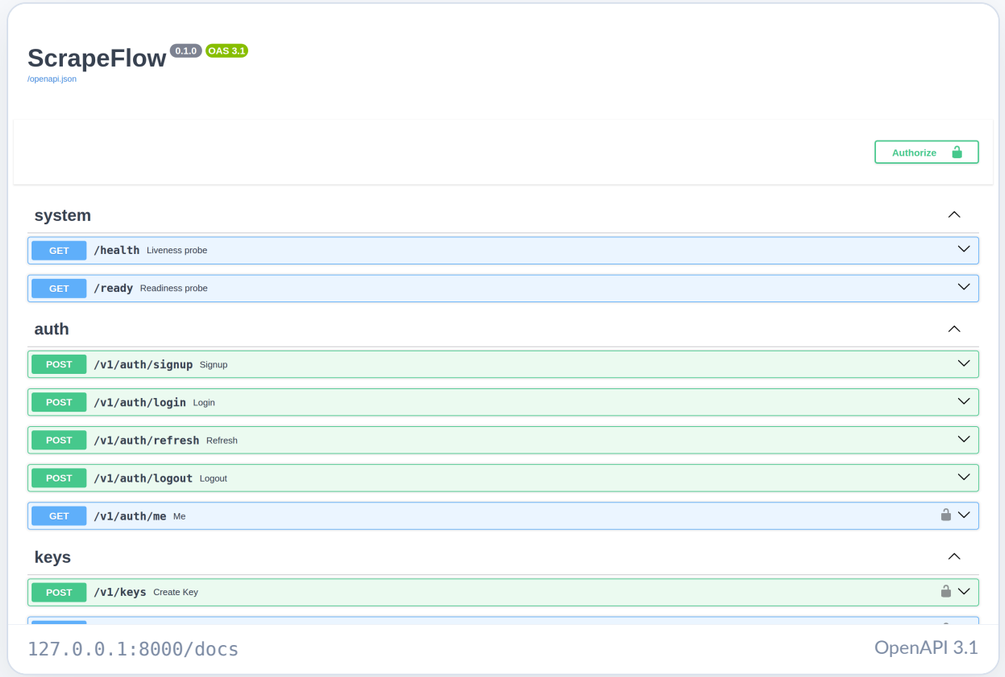
- **screenshot.png:** visual proof for humans and change detection.
- **page.html:** the rendered DOM for offline re-parsing and debugging.
- **result.json:** the machine-readable extraction your pipeline consumes.

API SURFACE

29 documented routes, generated from the code

The OpenAPI schema is not hand-maintained: it is built from FastAPI routes and Pydantic models, then served at `/docs`.

GROUP	ENDPOINTS
System	`GET /health · GET /ready`
Auth	`POST /v1/auth/signup · login · refresh · logout · GET /v1/auth/me`
Scrape	`POST /v1/scrape`
Automation	`POST /v1/automate`
Jobs	`GET /v1/jobs · GET /v1/jobs/{id} · cancel · events · export`
Artifacts	`GET /v1/jobs/{id}/artifacts · GET /v1/artifacts/{id}/download`
Keys	`GET/POST /v1/keys · PATCH/DELETE /v1/keys/{id}`
Schedules	`CRUD · run · runs · history · export`
Webhooks	`CRUD · deliveries · rotate-secret`
Billing	`checkout · portal · plans · usage · webhook`



Async by design: creation endpoints return `202` and a job id. Clients poll or wait for a webhook — the HTTP connection is never held open for a browser run.

IN SHORT The schema is generated from the implementation, so documentation cannot silently drift.

INFRASTRUCTURE · LOCAL

Eleven services in one Compose file

One command builds and starts everything, waits for readiness, and prints the URLs.

SERVICE	IMAGE	PORT	RESPONSIBILITY
web	`apps/web Dockerfile`	`3000`	Next.js dashboard and BFF, production build
api	`apps/api Dockerfile`	`8000`	FastAPI control plane, OpenAPI, metrics
worker	`same backend image`	`9101`	Celery prefork workers, scrape + automation
beat	`same backend image`	—	Celery Beat scheduler, schedule file on tmpfs
migrate	`same backend image`	—	one-shot `alembic upgrade head`, exits 0
egress-proxy	`same backend image`	`8888`	validating HTTP/CONNECT proxy with IP pinning
postgres	`postgres:16-alpine`	`5432`	durable relational state, named volume
redis	`redis:7-alpine`	`6379`	broker, token buckets, idempotency
minio	`quay.io/minio/minio`	`9000/9001`	S3-compatible artifact store (s3mock fallback)
minio-init	`python helper`	—	creates the bucket, exits 0
selenium	`standalone-chrome:4.25.0`	`4444`	headless Chrome pool for real browser runs

VOLUMES
`pgdata · redisdata · miniodata`

NETWORK
single `app_net` bridge; only 3000/8000/9000/4444 bound to host

HEALTH
pg_isready, redis-cli ping, api /ready, web /login

IN SHORT Eleven services, one network, three volumes. Reset is safe because all state is named.

INFRASTRUCTURE · PRODUCTION

Same topology, one TLS edge, explicit runbook

`deploy/compose.prod.yml` reuses the backend image, adds Caddy 2.8 and removes host-exposed dev ports.

EDGE

Caddy 2.8

Automatic HTTPS certificates, reverse proxy to `web` and `api`, HTTP→HTTPS redirect. Only 80/443 are published.

DATA

Named volumes + backups

PostgreSQL, Redis and MinIO volumes persist across deploys. The runbook covers `pg_dump`/restore, volume backup and a tested rollback path.

DEPLOY

Image-based

`SCRAPEFLOW_IMAGE` selects the backend tag; `migrate` runs before API/worker start; rollback is a tag change plus a migration check.

Not started here: the production Caddy topology needs a real domain and DNS. It is committed and documented, but **not verified live** in this environment.

deploy/compose.prod.yml · services

```
caddy          # TLS + reverse proxy, ports 80/443
api            # uvicorn, no host port
web            # next start, no host port
worker         # celery prefork
beat           # scheduler
migrate        # alembic upgrade head (one-shot)
egress-proxy   # validating forward proxy
postgres       # 16-alpine + pgdata
redis          # 7-alpine + redisdata
minio          # S3 store + miniodata
minio-init     # bucket bootstrap
selenium       # standalone-chrome
```

RUNBOOK COVERAGE

- Start, stop, scale workers, rotate secrets.
- Backup and restore PostgreSQL; verify object storage.
- Rollback an image tag and confirm migrations.
- Incident triage: `/ready`, logs, queue depth, reaper.

IN SHORT Production changes the edge, not the application: same images, Caddy in front, no dev ports.

DEVELOPER EXPERIENCE

One command on Linux, macOS and Windows

Four entry points, identical subcommands. The macOS wrapper delegates to the shared Bash runner; Windows has a native PowerShell implementation.

ENTRY POINT	PLATFORM	BEHAVIOUR
scripts/run.sh	Linux	Shared implementation: start, logs, status, seed, fix-db-password, stop, reset
scripts/run-macos.sh	macOS	Checks Docker Desktop and Python, then `execs` the shared runner
scripts/run.ps1	Windows	Native PowerShell 5.1+: creates `.env` and the S3-mock override without Bash, Python or curl
scripts/run.cmd	Windows CMD	Launcher that runs `run.ps1` with `-ExecutionPolicy Bypass`

PowerShell 7.4.6 · exercised on Linux

```
PS> .\scripts\run.ps1 status
NAME                    STATUS
scrapeflow-api-1        Up (healthy)
scrapeflow-worker-1     Up
scrapeflow-web-1        Up (healthy)

PS> .\scripts\run.ps1 fix-db-password
==> Starting PostgreSQL
ALTER ROLE
role scrapeflow now matches DATABASE_URL
```

WHAT WAS VERIFIED

- `bash -n` passes for both Bash scripts, including in a real Bash 3.2 container.
- The macOS wrapper delegates correctly (run on Linux).
- PowerShell: syntax, PSScriptAnalyzer (no errors), `status`, `fix-db-password`, `.env` generation (64-char secrets, mode 600, no BOM), fake-Docker start path.
- `.gitattributes` keeps shell/YAML LF and Windows launchers CRLF.

Honest limit: native Windows PowerShell 5.1 + Docker Desktop and native macOS execution were **not available** on this host. They remain UNVERIFIED; the scripts target those environments and the docs say so.

IN SHORT The platform is a first-class target, not a footnote in the README.

QUALITY GATES

322 Python tests, 30 web tests, zero lint or type errors

Numbers below come from the final run recorded in `docs/presentation/evidence/`.

322
PYTHON TESTS PASSED

83.08%
BRANCH/LINE COVERAGE

30
VITEST TESTS PASSED

0
LINT / TYPE ERRORS

python gates

```
$ uv run ruff check apps packages tests
All checks passed!
$ uv run ruff format --check apps packages tests
137 files already formatted
$ uv run mypy apps packages
Success: no issues found in 94 source files
$ uv run pytest -q --cov=apps --cov=packages
Required test coverage of 70.0% reached.
Total coverage: 83.08%
322 passed, 3 skipped in 95.21s
```

WHY 3 SKIPS

- **Chrome binary test:** `CHROME_BINARY` not set on this host; the Grid path is covered by an injected-driver unit test.
- **Grid-only worker test:** requires a live Selenium Grid browser.
- **Locust import check:** Locust is in the `e2e` group and not installed here.

TEST SHAPE

- 39 Python test files: unit, integration, e2e and contract tests.
- Integration tests run against real PostgreSQL and Redis in disposable containers.
- Runner tests guard every platform entry point, so documentation cannot drift from scripts.
- Compose regression tests assert the worker keeps the shared backend environment.

IN SHORT Every skip is documented with a reason. A green run without that context would be misleading.

SCOPE DISCIPLINE

What is proven, and what is still an assumption

A production claim is only as good as the environment it ran in. This slide is deliberately split.

Verified on this host

EVIDENCE

- Full dev Compose stack: 11 services, PostgreSQL/Redis/S3 mock healthy, migrations exit 0.
- Real browser scrape through Selenium Grid: screenshot + HTML + JSON artifacts.
- Artifact download through the API's presigned redirect.
- JWT signup/login/refresh and the dashboard session.
- Celery Beat scheduling, reaper, retention and worker metrics on 9101.
- 322 Python tests, Ruff, format and mypy; 30 Vitest tests, lint, typecheck, production build.
- Playwright Chromium E2E: 5 passed, 2 skipped.
- Runner scripts exercised: Bash 3.2 syntax, PowerShell 7.4.6 behaviour.

Not verified here

EXTERNAL DEPENDENCY

- **Live Stripe checkout/portal** — no API keys or test account.
- **Official MinIO image and lifecycle** — the pinned tag returned 401; s3mock was used.
- **Production Caddy topology** — needs a real domain and DNS.
- **Commercial proxy gateway** — no upstream credentials.
- **Real external webhook receiver** — HMAC path is unit-tested only.
- **Hosted GitHub Actions** — workflow is config-validated, not run on GitHub.
- **Locust capacity numbers** — Locust not installed; profile shape tested only.
- **Native Windows / macOS** — scripts target them but were not executed there.

Read this as: the application logic, security boundaries and integrations are implemented and tested locally. Anything that needs a paid or external service is clearly flagged — never presented as already working in production.

IN SHORT The left column is evidence. The right column is a to-do list that needs external services.

NEXT STEPS

From verified local platform to production service

NOW → 2 WEEKS

Close the external gaps

- Stripe test-mode keys: checkout, portal, webhook replay.
- Point artifacts at real MinIO or S3 with lifecycle rules.
- Deploy Caddy on a staging domain and run the HTTPS journey.
- Wire one real webhook receiver and verify HMAC + retries.

2 → 6 WEEKS

Prove capacity

- Run Locust against the queue with realistic selector payloads.
- Measure browser concurrency limits per Grid node.
- Add queue-depth autoscaling for workers.
- Publish p50/p95 job duration and error budget.

6+ WEEKS

Product depth

- Change detection and diff notifications on schedules.
- Per-tenant proxy pools with health checks.
- SDKs generated from the OpenAPI schema.
- Team invitations and finer-grained roles.

ENGINEERING GUARDRAILS ALREADY IN PLACE

- Every new dependency goes through `uv` or `npm` with a committed lockfile.
- Every API change updates Pydantic schemas, and therefore OpenAPI.
- Every platform script has a test that fails if the commands drift.
- Unverified paths are recorded in the final report and this deck.

SUCCESS CRITERIA FOR THE NEXT MILESTONE

- A paid test subscription completes end to end in Stripe test mode.
- A staging HTTPS deployment passes the same Playwright journey.
- A Locust run produces a documented throughput and latency curve.
- All three platforms run the stack natively at least once.

IN SHORT The next milestone is not more features — it is proving the external paths and the capacity curve.

SF

THANK YOU

ScrapeFlow · project and technology stack

A scraping platform is a queue, a boundary and a receipt.

Queue the work so requests stay fast. Guard the boundary so user URLs cannot reach your network. Keep a receipt — artifacts, hashes, events and metrics — so every job can be explained later.

RUN IT

```
./scripts/run.sh
./scripts/run-macos.sh
.\scripts\run.ps1
```

READ IT

```
SETUP.md
docs/FINAL_REPORT.md
docs/runbook.md
```

VERIFY IT

```
uv run pytest -q
npm test && npm run build
npm run e2e
```

Repo: `web-scrapping-tool` · commit `0c6d96c` · 10 Nov 2026